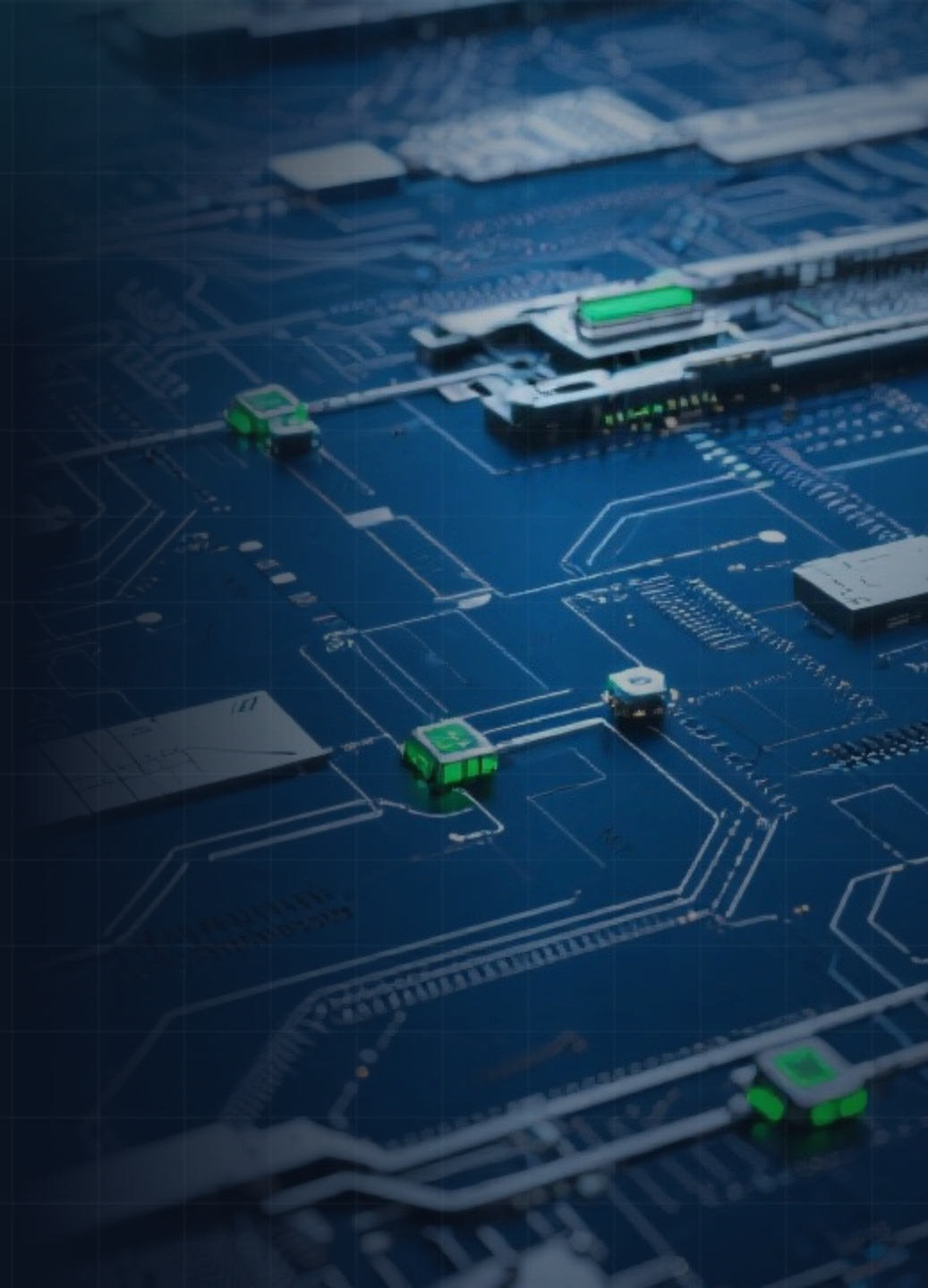
LARGE LANGUAGE MODELS: PRODUCTION AND DEPLOYMENT

PEFT, LoRA and QLoRA

Parameter-Efficient Fine-Tuning of Large Language Models

Adapt More. Train Less.

Lecture 4 | 60 minutes | PEFT → LoRA → QLoRA



TOPICS COVERED IN THIS LECTURE

- Full Fine-Tuning Cost
- What Is PEFT
- Frozen Parameters
- LoRA Idea
- LoRA Architecture
- Rank r
- Alpha α & Dropout
- Target Modules
- Quantization & QLoRA
- Full FT vs LoRA vs QLoRA

The Problem & PEFT

Full fine-tuning adapts everything — and that is exactly the problem. Freeze most of the model, train a small part.

IN THIS SECTION

1 Recall: Full Fine-Tuning
2 Why It Is Expensive
3 The PEFT Question
4 What Is PEFT?
5 Frozen Parameters
6 The PEFT Family

Recall: Full Fine-Tuning — and Why It Is Expensive

FULL FINE-TUNING

Pre-trained LLM

↓

Task-Specific Dataset

↓

Training

↓

Most / All Model Parameters Updated

↓

Specialized LLM

7B model ≈ 7 billion parameters potentially trainable

EVERY LAYER, EVERY WEIGHT

Layer 1	Layer 2	Layer 3	...	Layer N
TRAINABLE	TRAINABLE	TRAINABLE		TRAINABLE

TRAINING MEMORY ≠ MODEL WEIGHTS

Weights + Gradients + Optimizer States + Activations + Temporary Tensors + Framework Overhead

7 billion × 2 bytes ≈ 14 GB

— for FP16 raw parameter storage alone

! **Model Weight Size ≠ Total Training Memory.**

If the weights need ~14 GB, can I full-fine-tune comfortably on a 16 GB GPU?

→ **Not necessarily.**

The PEFT Question

Do we really need to update billions of parameters for every specialization?



DEFINITION

PEFT = **Parameter-Efficient Fine-Tuning** — techniques that adapt a pre-trained model while training only a relatively small number of parameters.

CONCEPT CHAIN

Large Pre-trained Model → Freeze most parameters →
Introduce / select small trainable components → Train those components →
Task-Specific Adaptation

BENEFITS

- Fewer trainable parameters
- Reduced training-memory requirement
- Smaller task-specific checkpoints
- Faster experimentation
- Easier multi-task specialization
- Multiple adapters can share one base model

• Frozen · Family

What Does Frozen Actually Mean? — The PEFT Family

FROZEN ≠ REMOVED

Suppose $w = 0.735$

DURING FINE-TUNING:

Forward pass	✓ Used
Gradient update	✗ Not updated

THEREFORE:

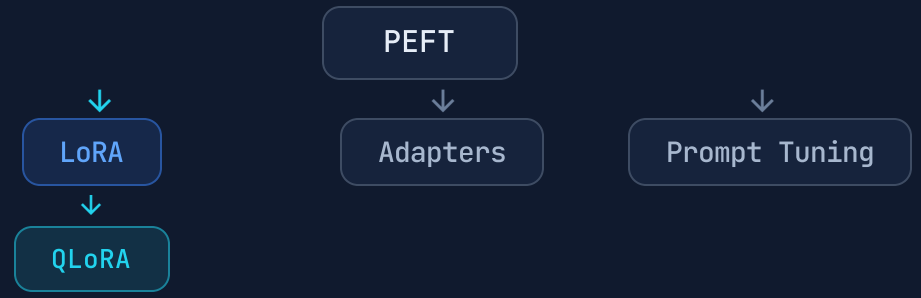
Before training 0.735 → After training 0.735

> *Frozen parameters still participate in computation. We simply prevent the optimizer from modifying them.*

If a parameter is frozen, is it removed from GPU memory?

→ **No. Remember this — it becomes the motivation for QLoRA.**

THE PEFT FAMILY



Other methods include Prefix Tuning and adapter-based methods.

FOR THIS COURSE

LoRA QLoRA

LoRA — Low-Rank Adaptation

Instead of changing a very large weight matrix directly, LoRA learns a **small correction** to that matrix.

IN THIS SECTION

1 Core Idea
2 Architecture
3 Why Low Rank
4 Matrix Dimensions
5 Parameter Count
6 Rank & Alpha
7 Dropout
8 Target Modules
9 What Gets Saved
10 Misconceptions

What Is LoRA? — The Main Idea

LoRA = Low-Rank Adaptation

ORIGINAL MODEL WEIGHTS — FROZEN

W_0 — the large pre-trained weight matrix. Used in every forward pass, never updated.

SMALL TRAINABLE MATRICES — LEARNED

A and B — small low-rank matrices. The only components that receive gradient updates.

> *Instead of changing a very large weight matrix directly, LoRA learns a small correction to that matrix.*

Full Fine-Tuning = rewrite much of a 500-page textbook

VS

LoRA = keep the textbook and add a small specialized supplement

ORIGINAL BEHAVIOR

$$y = W_0 x$$

during full fine-tuning W_0 itself changes

→

DESIRED UPDATE

$$W' = W_0 + \Delta W$$

W_0 original frozen weight · ΔW learned task-specific update

→

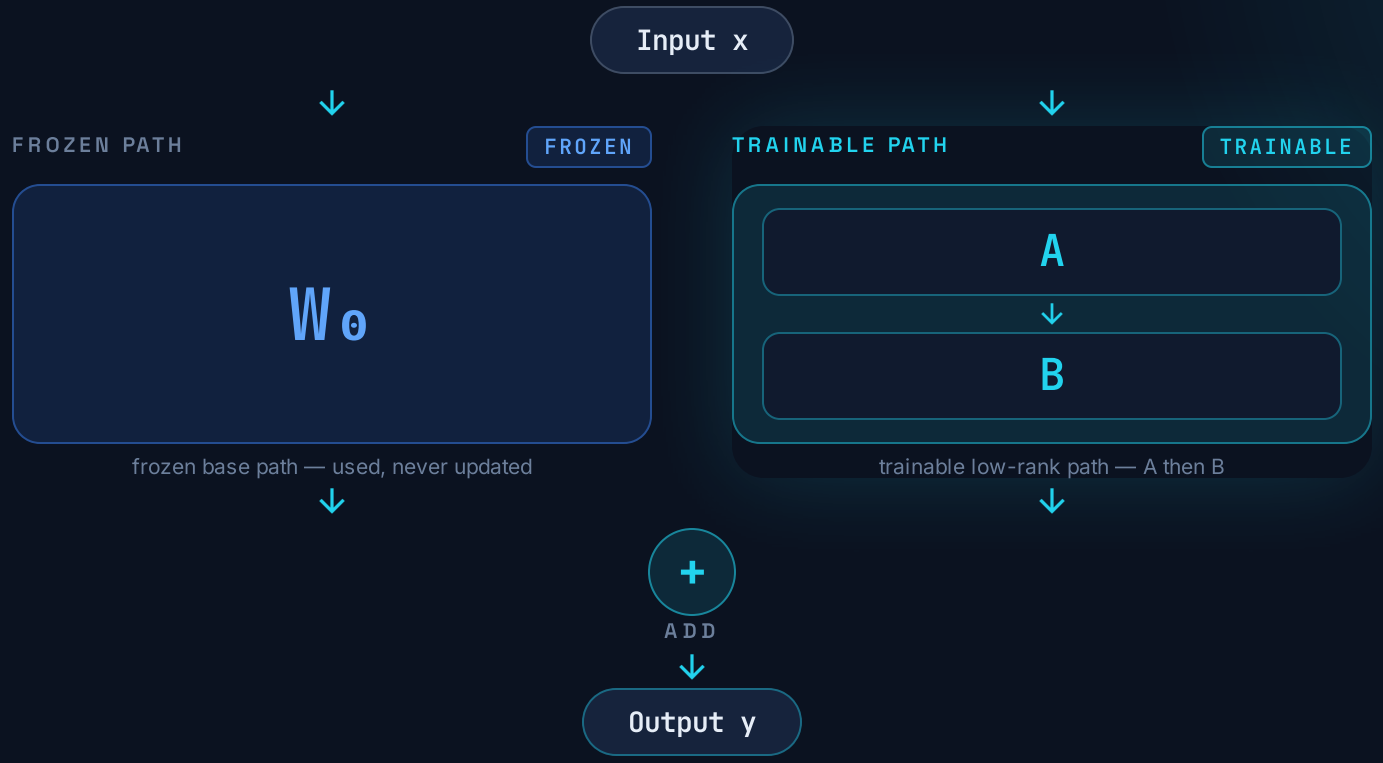
LORA TWIST

$$\Delta W = BA$$

A and B are much smaller matrices

LoRA does not directly train another huge ΔW. This is the core of LoRA.

LoRA Architecture



$$y = W_0x + (\alpha/r)\cdot BAx$$

W_0 frozen base weight

A, B trainable low-rank matrices

r rank

α scale

• Low Rank

Why Is It Called Low Rank? — Matrix Dimensions

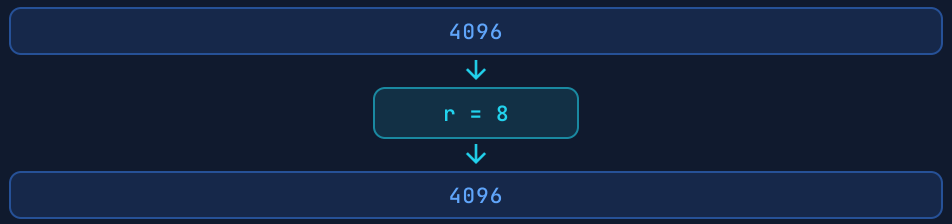
THE BOTTLENECK

original matrix huge: $W_0 \in R^{4096 \times 4096}$

instead of learning another 4096 × 4096 matrix, LoRA learns

A: 8 × 4096 | **B: 4096 × 8**

small value `r = 8` creates a narrow bottleneck



The update is constrained through a low-dimensional representation — that is the intuition behind low rank.

GENERAL FORM

$W_0 \in R^{d_out \times d_in}$	base
$A \in R^{r \times d_in}$	down-project
$B \in R^{d_out \times r}$	up-project
therefore $BA \in R^{d_out \times d_in}$	same shape as W_0

BA has the same final shape as W_0.

Why must BA have the same dimensions as W_0?

→ Because we are adding them: $W' = W_0 + BA$.

SECTION 02 · PARAMETER COUNT

Parameter Math

Parameter Count — and a 256× Example

FULL UPDATE

Full update parameters = $d_{out} \cdot d_{in}$

LORA UPDATE

LoRA parameters = $r \cdot (d_{in} + d_{out})$

SQUARE CASE · RATIO

if $d_{in} = d_{out} = d \rightarrow$ Full d^2 vs LoRA $2dr$

ratio $d^2 / (2dr) = d / (2r)$

WORKED EXAMPLE

4096 × 4096 · r = 8

1	Full matrix	4096 × 4096 = **16,777,216** parameters
2	LoRA	A 8 × 4096 = 32,768 · B 4096 × 8 = 32,768 · total = **65,536**
3	Reduction	16,777,216 / 65,536 = **256**

REDUCTION FACTOR

≈ 256×

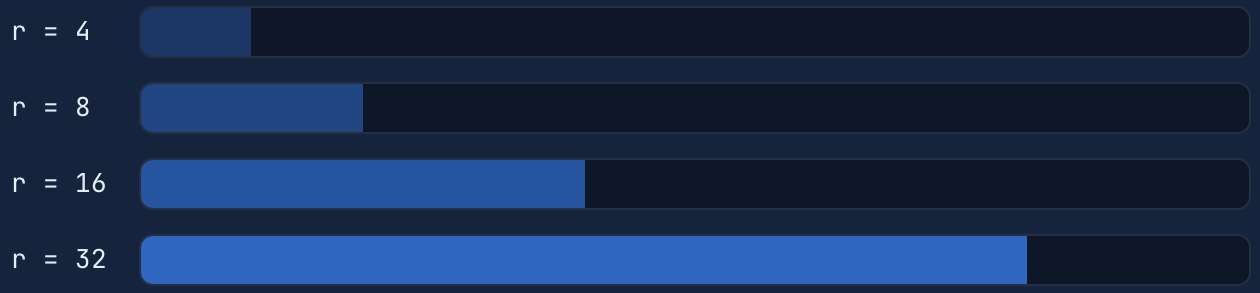
fewer trainable parameters

⚠ Do not tell students the entire model becomes 256× smaller — this calculation is for one targeted matrix.

LoRA Rank r

ADAPTATION CAPACITY

bottleneck width r



Bar length ∝ r · (d_{in} + d_{out}) — trainable parameters per targeted matrix.

AS r INCREASES →

- Adaptation capacity ↑
- Trainable parameters ↑
- Memory requirement ↑

BUT REMEMBER

Higher rank ≠ automatically better model.

Q **Would you always choose the largest possible rank?** → **No.** Rank is a hyperparameter selected using validation experiments.

LoRA Alpha α — and Dropout

ALPHA α

$$\Delta W = (\alpha / r) \cdot BA$$

- r controls the capacity of the adaptation
- α controls the scale / strength of the LoRA contribution

VOLUME ANALOGY — LORA SIGNAL vs BASE



! Do not teach that α = 2r is a universal rule — it is only a common configuration pattern in some experiments.

DROPOUT

LoRA dropout applies dropout to the LoRA adaptation path during training.



regularization

help reduce overfitting

Like ordinary dropout, not every contribution is used on every training step.

Where Do We Attach LoRA? — What Gets Saved?

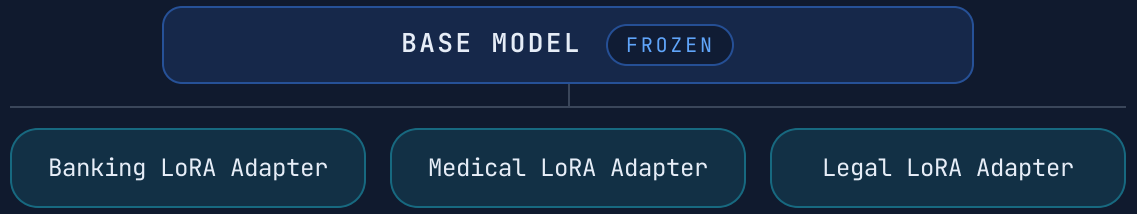
TARGET MODULES



Common LoRA targets may include `q_proj`, `k_proj`, `v_proj`, `o_proj` — and depending on architecture possibly `gate_proj`, `up_proj`, `down_proj`.

! Target module names depend on model architecture. Do not assume LoRA always goes only into q_proj and v_proj — that is configuration-specific.

WHAT GETS SAVED



> *We don't necessarily save another entire base model for every task.*

Shared base + small task-specific adapters — a major operational advantage.

SECTION 02 · LIMITS & MISCONCEPTIONS

Misconceptions

What LoRA Does NOT Do

LORA DOES

- ✓ Reduce trainable parameters
- ✓ Reduce training resource requirements
- ✓ Reduce task-specific checkpoint size
- ✓ Simplify multi-adapter specialization

LoRA does NOT automatically…

- ✗ Turn a 7B model into a 7M model
- ✗ Remove the base model
- ✗ Guarantee faster inference
- ✗ Guarantee better accuracy
- ✗ Eliminate GPU-memory requirements

Frozen ≠ Free.

This is the perfect transition to QLoRA. →

03

QLoRA — Quantized Adaptation

LoRA reduced how many parameters we train. Quantization reduces how much memory the frozen base occupies. **QLoRA combines both.**

AGENDA — SECTION 03

01 Why QLoRA?
02 What Is Quantization?
03 QLoRA Architecture
04 NF4
05 Double Quantization
06 Storage ≠ Computation
07 Method Comparison
08 Choosing a Method

SECTION 03 · QLORA MOTIVATION

Quantization

Why QLoRA? — What Is Quantization?

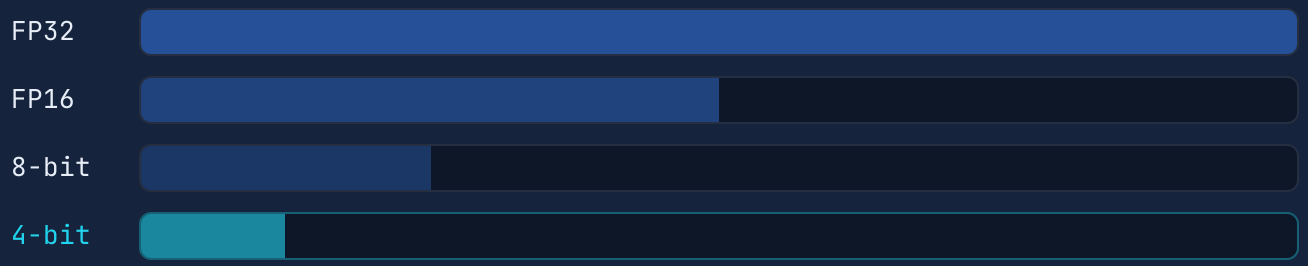
WHERE WE ARE

Too many TRAINABLE parameters → LoRA ✓ solved

Frozen 7B base model still occupies GPU memory → reduce it? → Quantization → QLoRA

Quantization: represent model weights using fewer bits.

RAW STORAGE BY PRECISION



Approximate raw storage for 7 billion weights.

CAVEAT

Simplified raw weight-storage estimates. **Actual GPU usage is higher.**

THE DIFFERENCE COMES FROM

activations · optimizer states · runtime overhead

What Is QLoRA?

DEFINITION

QLoRA = **Quantized** Low-Rank Adaptation.

CONCEPT STACK

4-bit Quantized Base Model — FROZEN

+

LoRA Matrices — TRAINABLE

> LoRA reduces **how many parameters** we train.

> Quantization reduces **how much memory the frozen base weights occupy**.

> QLoRA **combines both ideas**.

LoRA ARCHITECTURE

Higher-precision frozen base

+

Trainable LoRA

vs

QLoRA ARCHITECTURE

4-bit quantized frozen base

+

Trainable LoRA

LoRA → parameter-efficient training; QLoRA → parameter-efficient training + **memory-efficient base representation**

Advanced QLoRA Concepts — Precision Clarification

CONCEPT 01

NF4 (NormalFloat4)

A 4-bit representation designed to work well with neural-network weight distributions.

CONCEPT 02

Double Quantization

Quantizes some of the quantization-related constants too, saving additional memory.

CONCEPT 03

Paged Optimizers

Help manage temporary memory pressure and spikes.

You do not need to implement these from scratch. You need to know why they exist.

CLARIFICATION

Storage Precision ≠ Computation Precision

Quantized weights may be stored compactly and converted / dequantized as required for supported computation.

4-bit storage does not mean all neural-network arithmetic is simply 4-bit integer arithmetic.

SECTION 03 · QLORA

Comparison

Full FT vs LoRA vs QLoRA

PROPERTY	FULL FT	LORA	★ QLORA
Base weights	Trainable	Frozen	**Frozen + quantized**
Trainable parameters	Very high	Low	**Low**
Base-weight memory	High	High-ish	**Lower**
Task checkpoint	Large	Small	**Small**
Flexibility	Maximum	High	**High**
Training resources	Highest	Lower	**Lowest of these typically**
Good for limited GPU	Poor	Better	**Often best**

Avoid saying QLoRA is universally superior — it depends on constraints.

Decision

Which One Would You Choose?

A SITUATION A

- Large GPU cluster
- Maximum adaptation flexibility

→ `Full Fine-Tuning` may be considered

B SITUATION B

- Base model fits comfortably
- Limited training resources
- Multiple tasks

→ LoRA

C SITUATION C

- Base model itself is difficult to fit in memory

→ QLoRA

If the model fits for inference but not for full training, which direction would you explore? → PEFT / LoRA / QLoRA.

Practice & Wrap-Up

From concept to laboratory — pipelines, your own lab experiment, misconceptions, and the one-sentence takeaway.

LECTURE 4 · PEFT, LORA AND QLORA

AGENDA

01 LoRA Pipeline
02 QLoRA Pipeline
03 Lab Connection
04 Helpdesk Case
05 Misconception Quiz
06 Numerical Challenge
07 Summary
08 Bridge to Lecture 5

Practical LoRA & QLoRA Pipelines

LORA PIPELINE

13 STEPS

1. Choose Base Model
2. Prepare Dataset
3. Load Tokenizer
4. Load Base Model
5. **Freeze Base Parameters**
6. **Configure LoRA**
7. Choose Target Modules
8. Choose r, alpha, dropout
9. Fine-Tune
10. Validate
11. **Save Adapter**
12. Base + Adapter
13. Inference

QLORA PIPELINE

9 STEPS

1. Choose Base Model
2. **Load Base in 4-bit**
3. Prepare Quantized Model for Training
4. Attach LoRA
5. Train LoRA Parameters
6. Validate
7. Save Adapter
8. Quantized Base + Adapter
9. Inference

Key difference: the base model is loaded in 4-bit up front — everything else matches.

This is exactly what we will implement in the laboratory.

Your Lab Experiment — University Helpdesk Case

CONNECTION TO YOUR LAB

BASE MODEL `Qwen2.5-0.5B-Instruct`

↓

LoRA

↓

Trainable Parameters ≈ tiny fraction of total model

↓

Task-specific behavior

If only around a fraction of one percent of parameters are trainable, is the complete model now only that size?

→ **No. The large base model still exists.**

UNIVERSITY HELPDESK CASE STUDY

INPUT

"My semester fee was deducted but the portal still shows unpaid."

REQUIRED RESPONSE

```
Department: Accounts
Category: Payment Reconciliation
Priority: High
Action: Verify transaction reference
```

General LLM → **LoRA** → Learns university-specific response behavior

If current policies are also required: **RAG + LoRA**

RAG → Current Knowledge | **LoRA** → Specialized Behavior

SECTION 04 · QUIZ & DRILL

Quiz · Drill

Misconception Quiz — Numerical Challenge

TRUE / FALSE ROUND

6 STATEMENTS

LoRA trains the whole base model.	✕ FALSE
Frozen parameters are removed.	✕ FALSE
LoRA automatically makes inference much faster.	✕ FALSE
QLoRA usually combines a quantized base with LoRA adapters.	✓ TRUE
Higher rank is always better.	✕ FALSE
A 7B model with 7M trainable LoRA parameters becomes a 7M model.	✕ FALSE

90-SECOND CHALLENGE

$W \in R^{2048 \times 2048}, \quad r = 8$

Full matrix: $2048^2 = 4{,}194{,}304$

LoRA: $8(2048) + 2048(8) = 16{,}384 + 16{,}384 = 32{,}768$

Reduction: $4{,}194{,}304 / 32{,}768 = 128$

128× fewer trainable parameters
for that matrix

Have one student solve it verbally.

SECTION 04 · WRAP-UP

Summary

Final Summary — Bridge to Lecture 5

FULL FINE-TUNING
Train most / all of the large model

→

LORA
Freeze the base · Train a small low-rank adaptation

→

QLORA
Quantize the frozen base · Train the small LoRA adaptation

LORA UPDATE RULE

$$W' = W_0 + (\alpha/r)\cdot BA$$

`Full Fine-Tuning → Train the model` `LoRA → Train the adaptation` `QLoRA → Compress the base + train the adaptation`

🚀 BRIDGE TO LECTURE 5 — LLM INFERENCE AND INFERENCE LATENCY

> *LoRA and QLoRA helped us make training more practical. But what happens after training, when thousands of users begin sending requests to our model?*

`User Request` → `LLM Inference` → `??? milliseconds` → `Response`

NEXT LECTURE: Inference · Latency · Throughput · Memory · Batching · Production Optimization